

1:

a)

userInput

firstD

secondD

thirdD

b)

1stValue

Illegal because it starts with a 1

my variable

Illegal because it has a space

total-cost

Illegal because it has a hyphen

class

Illegal because it is a reserved key word

2:

a)

int numBeads;

numBeads = 5;

b)

int numBeads = 5;

3:

a)

13

b)

11

4:

a)

int

b)
Float

c)
int

d)
float

e)
boolean

f)
String

5

a primitive data type is a concrete, built-in building block defined by the programming language that dictates exactly how data is stored in memory, while an **abstract data type (ADT)** is a theoretical concept or specification that defines data purely by its behavior and allowed operations, completely ignoring how it is implemented or stored

11

A) `y = j * k;`

- Rewritten Statement: `y = (int)(j * k);` or `y = (int)Math.round(j * k);`
- Explanation: Multiplying an `int` (`j`) and a `double` (`k`) results in a temporary `double` value (`5 * 1.6 = 8.0`). Since `y` is an `int`, you cannot directly assign a `double` to it without causing a compile-time truncation error.
 - Use `(int)(j * k)` if you simply want to chop off any decimal pieces (truncation).
 - Use `(int)Math.round(j * k)` if you need the result mathematically rounded to the nearest whole number before converting.

B) `z = j * k;`

- Rewritten Statement: `z = j * k;`
- Explanation: Typecasting is not needed. The expression `j * k` automatically promotes the integer `j` to a `double` before performing the multiplication. The resulting value is a `double` (`8.0`), which perfectly matches the data type of the destination variable `z`.

C) `z = k * k;`

- Rewritten Statement: `z = k * k;`
- Explanation: Typecasting is not needed. Both `k` and `k` are already `double` variables. Multiplying them results in a `double` (`2.56`), which can be directly assigned to the `double` variable `z` without any loss of precision.

D) `j = k;`

- Rewritten Statement: `j = (int)k;` or `j = (int)Math.round(k);`
- Explanation: You are trying to assign a `double` value (`1.6`) into an `int` variable (`j`). This requires explicit downcasting because an integer cannot hold decimal data.
 - Direct casting `(int)k` will truncate the decimal, assigning `1` to `j`.
 - Using `(int)Math.round(k)` will round `1.6` up to `2.0` and then store `2` in `j`.

E) `k = j;`

- Rewritten Statement: `k = j;`
- Explanation: Typecasting is not needed. This is an automatic widening conversion (or implicit casting). An `int` value (`5`) can easily fit into a `double` variable (`k`) without losing any information, transforming into `5.0` automatically.